Chapter 2: The finite element method

Solutions to the exercises

Exercise 1: Conjugate gradients and preconditioners

1. $K = \begin{pmatrix} 3 & 2 \\ 2 & 6 \end{pmatrix}$, $F = \begin{pmatrix} 2 \\ -8 \end{pmatrix}$, $U_0 = (-2, -2)$; the solution is $U = (2, -2)$.

Steepest descent, $U_{k+1} = U_k + \alpha_k\, r_k$ with $\alpha_k = r_k^T r_k / r_k^T K\, r_k$:

$$\begin{aligned} r_0 &= F - KU_0 = (12, 8), & \alpha_0 &= \tfrac{208}{1200} = \tfrac{13}{75}, & U_1 &= (\tfrac{2}{25}, -\tfrac{46}{75}) \approx (0.080, -0.613), \\ r_1 &= r_0 - \alpha_0 K r_0 = \tfrac{112}{75}\,(2, -3), & \alpha_1 &= \tfrac{13}{42}, & U_2 &= (\tfrac{226}{225}, -2) \approx (1.004, -2.000), \end{aligned}$$

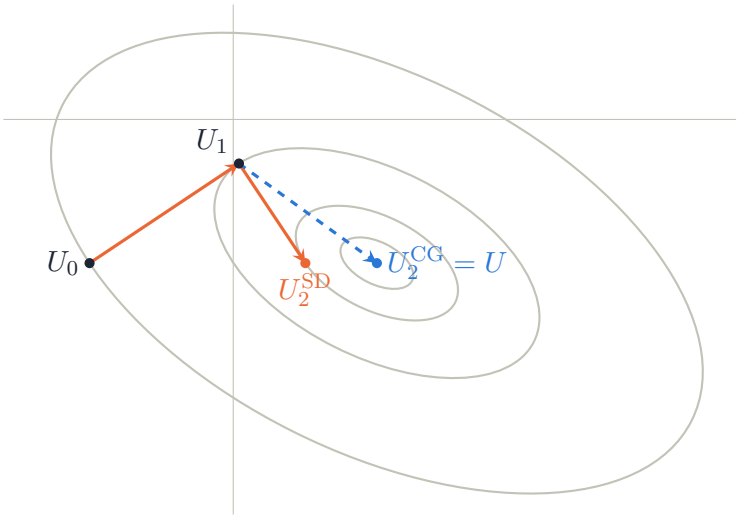
with $Kr_0 = (52, 72)$ and $Kr_1 = \frac{112}{75}\,(0, -14)$. The residuals are orthogonal, $r_0^T r_1 = 0$: each step turns by 90°, and after two steps the error is still $|U - U_2| = 1.0$.

CG starts with the same step ($p_0 = r_0$), so U_1 and r_1 are those above. Then

$$\beta_0 = \frac{r_1^T r_1}{r_0^T r_0} = \frac{784}{5625}, \qquad p_1 = r_1 + \beta_0\, p_0 = \tfrac{1456}{5625}\,(18, -13) \approx (4.659, -3.365),$$
$$\alpha_1 = \frac{r_1^T r_1}{p_1^T K\, p_1} = \frac{75}{182}, \quad U_2 = U_1 + \alpha_1\, p_1 = (2, -2), \qquad r_2 = 0.$$

CG reaches the solution after $n = 2$ steps. The directions are K-conjugate: $K(18, -13) = (28, -42)$ and $p_0^T K\, p_1 \propto (12, 8) \cdot (28, -42) = 0$.

The level sets of $\Phi(U) = \frac{1}{2} U^T K U - U^T F$ are ellipses around U with the axes along the eigenvectors of K: $\lambda = 2$ for $(2, -1)$ (long axis) and $\lambda = 7$ for $(1, 2)$, condition number 3.5. Steepest descent (orange) zigzags, CG (blue) reaches the centre in two steps:



2. Iterations until the residual has dropped by 10^{10} (script below):

N	n	none	Jacobi	ILU	AMG
8	81	9	9	8	4
16	289	30	30	13	5
32	1089	65	65	24	6
64	4225	131	131	41	6
128	16641	264	264	76	6
256	66049	532	532	148	6
512	263169	1074	1074	292	6

- **Jacobi** changes nothing. On this mesh of right-angled triangles every interior row of K has the diagonal entry 4 (row of node 12 in example 3), so $D = 4I$ on the coupled unknowns and $D^{-1}K = K/4$ has the same condition number; CG produces the same iterates. (The Dirichlet rows are decoupled by `assemble_system` and do not matter.) Jacobi helps only if the diagonal varies, e.g. on graded meshes or with varying coefficients.
- **ILU** needs about 3.7 times fewer iterations, but the number still doubles with every halving of h: the condition number is reduced by a constant factor, it still grows like h^{-2}, and the iterations like $1/h = \sqrt{n}$ in 2D.
- **AMG**: the number of iterations stays constant, the work is proportional to n.

```
# Chapter 2, exercise 1.2: CG iterations for the heated plate with different preconditioners (example 5)
from fenics import *

set_log_level(LogLevel.WARNING)

preconditioners = ['none', 'jacobi', 'ilu', 'amg']
print('   N        n' + ''.join('%9s' % p for p in preconditioners))
for N in [8, 16, 32, 64, 128, 256, 512]:
    mesh = RectangleMesh(Point(-1.0, -1.0), Point(1.0, 1.0), N, N)
    V    = FunctionSpace(mesh, 'Lagrange', 1)
    u, v = TrialFunction(V), TestFunction(V)
    bc   = DirichletBC(V, Constant(0.0), 'on_boundary')
    K, F = assemble_system(inner(grad(u), grad(v))*dx, Constant(1.0)*v*dx, bc)

    iterations = []
    for preconditioner in preconditioners:
        uh = Function(V)
        solver = KrylovSolver(K, 'cg', preconditioner)
        solver.parameters['relative_tolerance'] = 1e-10
        iterations.append(solver.solve(uh.vector(), F))
    print('%4d  %7d' % (N, V.dim()) + ''.join('%9d' % k for k in iterations))

# the diagonal of K: 4 at the interior nodes (h-independent), 1 in the Dirichlet rows set by assemble_system
mesh = RectangleMesh(Point(-1.0, -1.0), Point(1.0, 1.0), 16, 16)
V    = FunctionSpace(mesh, 'Lagrange', 1)
u, v = TrialFunction(V), TestFunction(V)
K, F = assemble_system(inner(grad(u), grad(v))*dx, Constant(1.0)*v*dx, DirichletBC(V, Constant(0.0), 'on_boundary'))
print('distinct diagonal entries of K (N = 16):', sorted(set(round(d, 12) for d in K.array().diagonal())))
```